CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler `0x` önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Mehmet Batuhan Özer
Öğrenci numarası (SID)	5001230025
E-posta	`batuhanozer1042@gmail.com`
Seçilen görev (G1–G5)	G5 — Run-length sıkıştırma
Seçilen ISA ve sürümü (ör. RV32IM)	x86-64 (Microsoft x64 ABI, PE/COFF)
Derleyici ve sürümü	clang-cl, LLVM 21.1.8 (MSVC 14.44.35207)
Simülatör / araçlar	Visual Studio 2022 (debugger), `llvm-objdump` 21.1.8; CMake 3.31.6 + Ninja 1.12.1
Git deposu adresi	`https://github.com/MBatuhanOzer/Computer_Organization_Project`

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001230025	**5**
D2	SID'in son iki hanesi	5001230025	**25**
S	SID'in hanelerinin toplamı	5+0+0+1+2+3+0+0+2+5	**18**
N	24 + (D2 mod 25)	24 + (25 mod 25) = 24 + 0	**24 eleman**
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d = 5 satırı	**256 byte / 16 byte / 4-way set-assoc. / 16 line / 4 set**

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
Write policy	d çift → write-back, d tek → write-through	d = 5 (tek)	**write-through**
k	(S mod 7) + 3	(18 mod 7) + 3 = 4 + 3	**7**
Interrupt türü	S çift → Timer, S tek → I/O	S = 18 (çift)	**Timer**
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 × 5	**1,5 GHz** (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T2 = 20 + 5 × 1	**1 cycle / 25 cycle** (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + 4	**6 çekirdek** (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	G5: R = 2 + (SID mod 3); 5001230025 mod 3 = 0	**R = 2**, B = 0

Türetilmiş: $\tau = 1/f = 1/(1{,}5\ \text{GHz}) = \mathbf{0{,}667}$ ns (varsayım). Tutarlılık: 256 byte / 16 byte = 16 line; 16 line / 4 yol = 4 set.

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	**4 bit**	line boyu 16 byte $= 2^4$; adres[3:0]
Line (r) veya set (d)	**2 bit**	set sayısı $4 = 2^2$; adres[5:4]
Tag ($s - r$ veya $s - d$)	**26 bit**	$s + w = 32 \Rightarrow s = 28$; $28 - 2$; adres[31:6]
Toplam	**32 bit**	$26 + 2 + 4$

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☐ G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☐ G4 Sıralama ve medyan
- ☒ **G5 Run-length sıkıştırma**

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

Program üç global dizi üzerinde çalışır: `v[24]` LCG ile üretilen $\{0, 1\}$ sembollerini, `enc[24]` kodlanmış (değer, tekrar) çiftlerini, `dec[24]` ise geri açılmış diziyi tutar. `rle_encode(n)` `v` dizisini tek geçişte tarar; dış döngü her koşu için bir kez döner, iç döngü aynı sembolün tekrar sayısını sayar ve her koşu `(v[i] << 8) | run` biçiminde tek bir 32-bit word'e paketlenerek `enc` dizisine yazılır, böylece shift, OR ve dizi store'u aynı satırda görünür hale gelir. `rle_decode(len)` bu paketleri `>> 8` ve `& 0xFF` ile ayrıştırıp `dec` dizisine açar; her iki fonksiyon da parametre alıp değer döndürür. Doğrulama iki aşamalıdır: önce eleman sayısı (`out == N`), sonra üçüncü geçişte eleman değerleri karşılaştırılır. Veri yerleşimi bilinçli olarak üç ayrı global dizi şeklinde seçilmiştir: üç dizi 15 cache line'a karşılık gelir, 256 byte'lık cache ise 16 line tutar; bu,

4-way set'lerden tam birinin taşmasını sağlayarak Aşama 4'te hem spatial locality'nin hem de conflict miss'in aynı iz üzerinde gözlenmesine imkân verir.

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloğa yapıştırılır (25–60 satır):

```c
// Name: Mehmet Batuhan Ozer
// Student Number: 5001230025
// Gorev G5: Run-Length Encoding (RLE)
// Personalized parameters: N = 24, R = 2, B = 0

#define N 24                    // 24 + (D2 mod 25) = 24 + (25 mod 25)
#define R 2                     // 2 + (SID mod 3) = 2 + 0
#define B 0                     // task definition

typedef unsigned int u32;       // 32-bit unsigned
typedef int s32;                // 32-bit signed

const int SID = 1230025;        // Y6: last 9 digits of 5001230025 (exceeds 2^31-1). Kept as const int exactly as the assignment specifies; s32 is a typedef of int anyway.

s32 v[N];                       // Y1: Original input array (global, 3 passes)
s32 enc[N];                     // RLE Encoded Array
s32 dec[N];                     // Decoded Array (Must be the same as the original array after decoding)

s32 enc_len = 0;                // Y7: number of runs produced
s32 ratio_x100 = 0;             // Y7: 100 * enc_len / N
s32 decode_ok = 0;              // Y7: 1 if dec[] matches v[]

// Y3: takes a parameter, returns a value. Encodes v[0..n-1] into enc[], returns the run count
s32 rle_encode(s32 n)
{
    s32 i = 0;
    s32 k = 0;
    while (i < n) {                         // Y1: 2nd pass over v
        s32 run = 1;
        while (i + run < n && v[i + run] == v[i]) { run++; }   // Y2: conditional branch
        enc[k] = (v[i] << 8) | run;         // Y4: shift and OR, Y5: store
        k++;
        i += run;                           // Y4: addition
    }
    return k;
}

// Y3: takes a parameter, returns a value. Expands enc[0..len-1] into dec[], returns element count
s32 rle_decode(s32 len)
{
    s32 o = 0;
    for (s32 j = 0; j < len; j++) {
        s32 value = enc[j] >> 8;            // Y4: shift
        s32 run = enc[j] & 0xFF;            // Y4: AND
        for (s32 t = 0; t < run; t++) {
            dec[o++] = value;               // Y5: store
        }
    }
    return o;
}

int main(void)
{
    // LCG input generation: x0 = SID, x = 1664525*x + 1013904223 (mod 2^32)
    u32 x = (u32)SID;                       // Y6: SID seeds the generator
    for (s32 i = 0; i < N; i++) {           // Y1: 1st pass over v
        x = 1664525u * x + 1013904223u;     // Unsigned 32-bit overflow is defined behaviour in C so it gives mod 2^32 automatically
        v[i] = (s32)((x >> 16) % R) + B;    // Y5: store to array
    }
    enc_len = rle_encode(N);              // Y3: parameterised call, value returned
    s32 out = rle_decode(enc_len);      // Y3: parameterised call, value returned
    ratio_x100 = (100 * enc_len) / N;       // Y4: multiplication
    decode_ok = (out == N);
    for (s32 i = 0; i < N; i++) {           // Y1: 3rd pass over v
        if (dec[i] != v[i]) {               // Y2: conditional branch
            decode_ok = 0;
```

```
67          }
68      }
69      return 0;
70  }
```

Tek dosya, C99, 54 satır kod (toplam 70 satır; 9 boş ve 6 yalnızca yorum içeren satır hariç). `clang -std=c99 -pedantic-errors -Wall -Wextra` ile uyarısız derlenir. Uzun yorum satırları listede ↪ ile sarılmıştır.

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	**15**; 56, 28, 64	`s32 v[N];` — `v` üzerinde üç geçiş: s. 56 yazma, s. 28 okuma/kodlama, s. 64 doğrulama
Y2	Koşullu dallanma	**30**; 65	s. 30 `v[i+run] == v[i]` koşu uzunluğunu belirler; s. 65 `if (dec[i] != v[i])`
Y3	Parametreli, değer döndüren fonksiyon çağrısı	**24**, 39 (tanım); **60**, 61 (çağrı)	`s32 rle_encode(s32 n)` ve `s32 rle_decode(s32 len)`; ikisi de parametre alır, değer döndürür ve dönen değer çağrı yerinde bir değişkene atanır
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	**31**, 33, 43, 44, 62	s. 31 `<< 8` ve `|`; s. 33 `i += run`; s. 43 `>> 8`; s. 44 `& 0xFF`; s. 62 `100 * enc_len`
Y5	Dizi elemanına store	**58**, 31, 46	`v[i] = ...` (s. 58), `enc[k] = ...` (s. 31), `dec[o++] = ...` (s. 46)
Y6	`const int SID` ve hesapta kullanımı	**13** (tanım), **55** (kullanım)	`const int SID = 1230025;` (numara $2^{31} - 1$'i aştığı için son 9 hane); LCG'nin x_0 tohumudur, dolayısıyla tüm `v` dizisini ve üç Y7 çıktısını belirler
Y7	Sonucun tutulduğu global değişken(ler)	**19, 20, 21**	`enc_len`, `ratio_x100`, `decode_ok`

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
`enc_len`	`0x140075C60`	**11** (`0x0000000B`)	`ss_globals.png`
`ratio_x100`	`0x140075C64`	**45** (`0x0000002D`)	`ss_globals.png`
`decode_ok`	`0x140075C68`	**1** (`0x00000001`)	`ss_globals.png`
`v[24]`	`0x140075C70`	`0001 0001 0000 0111 0010 1110`	`ss_globals.png`
`enc[0..10]`	`0x140075CD0`	`0x0003 0x0101 0x0003 0x0101 0x0005 0x0103 0x0002 0x0101 0x0001 0x0103 0x0001`	`ss_globals.png`
`dec[24]`	`0x140075D30`	`v` ile birebir aynı	`ss_globals.png`

Değerler Visual Studio 2022 Watch penceresinden okunmuştur; pencere onlu taban kullanır (`257` = `0x0101`, `259` = `0x0103`) ve `enc`'in tanımlı 24 elemanının tamamını gösterir, yalnızca ilk `enc_len` = 11 tanesi anlamlıdır. `ratio_x100` = $\lfloor 100 \times 11/24 \rfloor = 45$; yönergedeki $R = 2$ için "yaklaşık 50" beklentisine göre fark, $N = 24$'ün küçüklüğünden kaynaklanan örneklem dalgalanmasıdır (beklenen koşu sayısı 12, ölçülen 11).

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	s. 49 (`return o;`) ve s. 61–64
Başlangıç ve bitiş adresi (hex)	`0x140001168` – `0x140001232` (toplam 68 byte)
Dinamik instruction sayısı	**17**
Hangi döngü iterasyonu (ör. i = 3)	`rle_decode`'dan tek dönüş; ardından doğrulama döngüsünün ilk testi ($i = 0$)

İçerik kontrolü: load (#1, 5, 10, 16), store (#4, 9, 14), ALU (#5 `imul`, #8 `idiv`, #12 `and`), koşullu branch (#17 `jge`), fonksiyon dönüşü (#3 `ret`). $k = 7$ interrupt'ı 7. instruction olan `cdq`'ya (`0x140001208`) denk gelir; kaydedilecek PC = `0x140001209`. Kernel'in tam instruction listesi ek belgededir (`M1_Adim0_Adim1.pdf`, Bölüm 2.6).

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____
Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (-O0; listenin tamamı depoda asm_O0.s):

Derleme: `cmake --preset od` (`/Zi /Od /Ob0`). Her derleme `listings/` altına derleyicinin C satır işaretli assembly'sini (`asm_O0.s`) ve linklenmiş exe'deki kendi fonksiyonlarımızın disassembly'sini (`objdump_O0.txt`) yazar. İkisi instruction instruction eşlenerek her C satırının gerçek adres aralığı bulunmuştur; 146 instruction'ın tamamı `listings/c2asm_O0.txt` dosyasındadır.

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
55	`u32 x = (u32)SID;`	`mov dword ptr [rsp+0x30], 0x12c4c9`	`0x14000118C`–`0x140001193`
57	`x = 1664525u * x + 1013904223u;`	`imul eax, dword ptr [rsp+0x30], 0x19660d; add eax, 0x3c6ef35f; mov dword ptr [rsp+0x30], eax`	`0x1400011A3`–`0x1400011B3`
28	`while (i < n) {`	test: `mov eax, [rsp+0x10]; cmp eax, [rsp+0x14]; jge 0x1400010C5` geri dönüş: `jmp 0x140001018`	`0x140001018`–`0x140001025` ve `0x1400010C0`–`0x1400010C4`
31	`enc[k] = (v[i] << 8) | run;`	`movsxd rcx, [rsp+0x10]; lea rax, [rip+..]` (v); `mov edx, [rax+4*rcx]; shl edx, 0x8; or edx, [rsp+0x8]; movsxd rcx, [rsp+0xc]; lea rax, [rip+..]` (enc); `mov [rax+4*rcx], edx`	`0x140001084`–`0x1400010A8`
60	`enc_len = rle_encode(N);`	`mov ecx, 0x18; call 0x140001000; mov [rip+..], eax` (enc_len)	`0x1400011DD`–`0x1400011EC`
62	`ratio_x100 = (100 * enc_len) / N;`	`imul eax, [rip+..], 0x64; mov ecx, 0x18; cdq; idiv ecx; mov [rip+..], eax`	`0x1400011FC`–`0x140001210`

Satır 28 iki ayrı adres aralığına düşer: döngü testi döngünün başında, geri dönüş `jmp`'i sonundadır; tek bir C satırı bitişik olmayan makine koduna karşılık gelebilir. Satır 30'daki `&&` koşulu -O0'da 23 instruction'a açılır: önce sınır testi (`jge`), ardından `cmp` + `sete` ile sonuç bir byte'lık stack değişkenine yazılıp tekrar okunur.

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
`.text`	`0x140001000`	395 750 (`0x609E6`)	Kendi kodumuz `0x140001000`–`0x140001272` (627 byte: `rle_encode`, `rle_decode`, `main`); geri kalanı statik CRT
`.rdata`	`0x140062000`	75 452 (`0x126BC`)	`SID` (`0x140062000`) ve CRT'nin salt-okunur verisi
`.data` / `.bss`	`0x140075000`	3 584 (`0xE00`)	`enc_len 0x140075C60`, `ratio_x100`, `decode_ok`, `v 0x140075C70`, `enc 0x140075CD0`, `dec 0x140075D30`. Sıfırla ilklenen globaller; PE'de ayrı `.bss` bölümü yoktur, linker bunları `.data`'ya katar.

Von Neumann ilkesi bu adreslerde nasıl görünüyor? Komutlar (`.text`, `0x140001000`) ve veriler (`.rdata 0x140062000`, `.data 0x140075000`) aynı tek ve doğrusal adres uzayındadır. Bu RIP-relative adreslemede doğrudan görülür: `0x1400011C6`'daki `lea rax, [rip+0x74AA3]`, `v`'nin adresini bir sonraki komutun adresine sabit bir uzaklık ekleyerek bulur (`0x1400011CD` + `0x74AA3` = `0x140075C70`); bu hesap ancak komut ve veri aynı adres uzayında durduğunda anlamlıdır.

Program `SID`'i `.rdata`'dan okumaz: `const int` olduğu için derleyici değeri satır 55'teki komutun içine immediate olarak gömer (`c9 c4 12 00`). `.rdata`'daki kopya yalnızca `SID`'in dış bağlantılı (external linkage) bir sembol olması nedeniyle üretilir.

2b. -O0 ve -O1 karşılaştırması (kernel için):

Kernel C satırlarıyla tanımlıdır (s. 49 `return o;` ve s. 61–64); `/O1` sütunu aynı C satırlarına ait instruction'ları sayar (`listings/c2asm_O1.txt`). Load ve store sayıları bütün bellek okuma/yazmalarını içerir; `ret` ve `pop` stack'ten okudukları için load sayılır.

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	17	18	Register allocation spill/reload'ları kaldırdı (`out` ve `i` artık register'da; s. 61: 1→0) ve döngü döndürüldüğü için ilk test silindi (`N` = 24 > 0 derleme anında bilinir; s. 64: 3→0). Buna karşılık `/ 24` bölmesi `cdq`+`idiv` yerine sabitle çarpma dizisine dönüştü (`imul` ile `0x2AAAAAAB`, `sar 34`; s. 62: 5→8) ve `rle_decode` dönüşte üç callee-saved register'ı geri yükledi (s. 49: 3→4). Net etki: daha az bellek erişimi, biraz daha fazla ama daha ucuz instruction.
Load sayısı	5	5	-O0: 3 spill okuma (`o`, `out`, `i`) + `enc_len` + `ret`. -O1: spill okuma 0; yerine 3 `pop` (callee-saved `rbx`, `rdi`, `rsi`) + `enc_len` + `ret`. Sayı aynı, içerik farklı: değişken trafiği tamamen kalktı.
Store sayısı	4	2	-O0'da `out` ve `i` stack'e yazılır (2 spill store); -O1'de register'da kalırlar. Kalan 2 store `ratio_x100` ve `decode_ok`'tur ve kaldırılamaz: sonuç global değişkende tutulmalıdır (Y7).
Kullanılan register sayısı	4	7	-O0 her değişkeni stack'te tuttuğu için yalnızca `rax`, `rcx`, `rdx`, `rsp` kullanır. -O1 değişkenleri register'larda yaşatır: `rax`, `rcx`, `rdx`, `rsp` + `rbx`, `rdi`, `rsi`. Bu üçü callee-saved olduğu için kullanılmaları onları kaydetme/geri yükleme maliyetini (`push`/`pop`) beraberinde getirir.

Bellek referansı toplamı (load + store): -O0'da 9, -O1'de 7. Stack üzerindeki değişken trafiği 5'ten 0'a iner; azalmanın tamamı register allocation'dan gelir.

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

Değişken uzunluklu x86-64'te "format", bir instruction'ın hangi alanları taşıdığıdır (prefix, REX, opcode, ModR/M, SIB, displacement, immediate). Format sütunu Intel SDM Cilt 2'nin "Op/En" sınıflandırmasını

kullanır. Seçilen 10 instruction programda kullanılan her alan türünü en az bir kez içerir: REX.W (#2), tek byte'lık opcode (çoğu), iki byte'lık `0F` opcode (#9), ModR/M.reg içinde opcode uzantısı (#2, #4), opcode içinde register (#8), ModR/M mod = 00 RIP-relative (#7), mod = 01 (#1, #4, #6), mod = 11 (#2, #9), SIB (#1, #4, #6), disp8 (#1, #4, #6), disp32 (#7), imm8 (#2, #7), imm32 (#4, #8), rel8 (#10), rel32 (#5) ve operandsız (#3). Instruction'ların 8'i kernel'den alınmıştır (#4 ve #5 hariç); doğrulama `listings/objdump_O0.txt` ile yapılır.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?

1	`0x140001168`	`mov eax, dword ptr [rsp+0x10]`	RM: `8B /r`	`op=10001011`
				`mod=01 reg=000`
				`rm=100`
				`ss=00 idx=100`
				`base=100`
				`disp8=00010000`
					`8B 44 24 10`	`8B, 44, 24, 10`	✓

2	`0x14000116C`	`add rsp, 0x18`	MI: `REX.W 83 /0 ib`	`REX=0100 W=1 R=0 X=0 B=0`
				`op=10000011`
				`mod=11 reg=000(/0)`
				`rm=100`
				`imm8=00011000`
					`48 83 C4 18`	`48, 83, C4, 18`	✓

3	`0x140001170`	`ret`	ZO: `C3`	`op=11000011`	`C3`	`C3`	✓

4	`0x14000118C`	`mov dword ptr [rsp+0x30], 0x12c4c9`	MI: `C7 /0 id`	`op=11000111`
				`mod=01 reg=000(/0)`
				`rm=100`
				`ss=00 idx=100`
				`base=100`
				`disp8=00110000`
				`imm32=0x0012C4C9`
					`C7 44 24 30 C9 C4 12 00`	`C7, 44, 24, 30, C9, C4, 12, 00 (imm32 LE)`	✓

5	`0x1400011F3`	`call 0x1400010D0 (rle_decode)`	D: `E8 cd`	`op=11101000` `rel32=0xFFFFFED8 (-296)`	`E8 D8 FE FF FF`	`E8, D8, FE, FF, FF (rel32 LE)`	✓

6	`0x1400011F8`	`mov dword ptr [rsp+0x28], eax`	MR: `89 /r`	`op=10001001`
				`mod=01 reg=000`
				`rm=100`
				`ss=00 idx=100`
				`base=100`
				`disp8=00101000`
					`89 44 24 28`	`89, 44, 24, 28`	✓

7	`0x1400011FC`	`imul eax, dword ptr [rip+0x74a5d], 0x64`	RMI: `6B /r ib`	`op=01101011`
				`mod=00 reg=000`
				`rm=101(RIP)`
				`disp32=0x00074A5D`
				`imm8=01100100`
					`6B 05 5D 4A 07 00 64`	`6B, 05, 5D, 4A, 07, 00, 64 (disp32 LE)`	✓

8	`0x140001203`	`mov ecx, 0x18`	OI: `B8+rd id`	`op=10111`
				`reg=001(ecx)`
				`imm32=0x00000018`
					`B9 18 00 00 00`	`B9, 18, 00, 00, 00 (imm32 LE)`	✓

9	`0x140001216`	`sete al`	M: `0F 94`	`esc=00001111`
				`op=10010100`
				`mod=11 reg=000`
				`rm=000(al)`
					`0F 94 C0`	`0F, 94, C0`	✓

10	`0x140001231`	`jge 0x14000126C`	D: `7D cb`	`op=01111101` `rel8=00111001 (+57)`	`7D 39`	`7D, 39`	✓

Doğrulama. Tablodaki 10 instruction'ın makine kodu, linklenmiş `build-od/rle.exe` üzerinde Visual Studio 2022 Disassembly penceresinde (*Show Code Bytes* açık) birebir doğrulanmıştır; ekran görüntüsü depoda `ss_m2_disasm.png` dosyasındadır. Pencere C satırlarını da gösterdiği için 2a'daki eşlemeyi de doğrular; stack değişkenleri PDB'den gelen adlarıyla görünür (ör. `[x]` = `[rsp+0x30]`, `[out]` = `[rsp+0x28]`).

Zorunlu instruction'ların tablodaki sıra numarası:

- ☒ SID'i yükleyen instruction(lar): # 4
- ☒ Koşullu branch: # 10

- ☒ Load: # 1
- ☒ Store: # 6
- ☒ Fonksiyon çağrısı: # 5

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

x86-64'te sabit tek instruction'a **sığar**, bölünmez ve düzeltme gerekmez. #4 (`C7 /0 id`) 32 bitlik bir immediate alanı taşır; `SID` $= 1\,230\,025 =$ `0x0012C4C9`, instruction'ın son dört byte'ına little-endian olarak yazılır: `C9 C4 12 00`. Bu, değişken uzunluklu encoding sayesinde mümkündür: instruction sabiti taşımak için 8 byte'a uzar. (`/O1`'de aynı sabit doğrudan register'a yüklenir: `mov eax, 0x12C4C9` = `B8 C9 C4 12 00`, yine tek instruction.)

Sabit 32 bit uzunluklu RV32I'de ise 32 bitlik bir sabit, 32 bitlik instruction'ın içine opcode ve register alanlarıyla birlikte sığmaz. Sabit ikiye bölünür: `lui` üst 20 biti, `addi` alt 12 biti yükler. `addi`'nin 12 bitlik immediate'i sign-extend edildiği için, alt 12 bitin en üst biti (bit 11) 1 olsaydı bu değer negatif yorumlanır ve sonuçtan 2^{12} düşerdi; bunu dengelemek için `lui` değerine $+1$ eklenirdi. `SID` için alt 12 bit `0x4C9` = `0100 1100 1001`, bit $11 = 0$ olduğundan düzeltme gerekmez: `lui 0x12C` ($\rightarrow$ `0x0012C000`) + `addi 0x4C9` = `0x0012C4C9`.

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

#10 `jge`, PC-relative'dir ve offset bir **sonraki** instruction'ın adresinden ölçülür (CPU branch'i execute ederken PC zaten bir sonraki instruction'ı gösterir).

1. Branch'in adresi: `0x140001231`; uzunluğu 2 byte (`7D` + rel8).
2. Bir sonraki instruction'ın adresi: `0x140001231` + 2 = `0x140001233`.
3. Hedef adres: `0x14000126C` (`xor eax, eax`, döngüden çıkış; s. 69).
4. Offset = hedef − sonraki = `0x26C` − `0x233` = `0x39` = $+57$.
5. $-128 \leq 57 \leq 127$ olduğu için 8 bitlik alan yeterli: rel8 = `0011 1001` = `0x39`. Machine code: `7D 39`.
6. Çalışırken: koşul (SF = OF, yani `i` ≥ 24) sağlanırsa PC $\leftarrow$ `0x140001233` + `0x39` = `0x14000126C`; sağlanmazsa PC `0x140001233`'te kalır.

Geriye doğru örnek (#5 `call`): sonraki adres `0x1400011F3` + 5 = `0x1400011F8`, hedef `0x1400010D0`; offset = `0x10D0` − `0x11F8` = −`0x128` = -296. 32 bitlik iki'ye tümleyen: `0xFFFFFED8`, little-endian `D8 FE FF FF`.

Ek — Öğretim üyesine sorular (gerekçeler ve ölçümler `M1_Adim0_Adim1.pdf`'de)

1. Aşama 4'ün izi `/O1` derlemesinden alınabilir mi? Ölçüm: `/Od`'de dinamik bellek referanslarının %83,4'ü stack frame'e aittir (961 / 1152), `/O1`'de ise %87,9'u gerçek dizi erişimidir (152 / 173). Yönergenin öğretim üyesi notları 6. maddesi bu alternatifi sunuyor.

2. Değişken uzunluklu instruction'larda instruction fetch kaç bus işlemi sayılacak? Kernel'in 17 instruction'ı 1–8 byte arasında değişip toplam 68 byte tutuyor. Aksi belirtilmedikçe Fig. 3.2 kavramsal modeline uygun olarak **instruction başına bir fetch bus işlemi** varsayacağım (adres = PC, data = instruction'ın kendisi).